

bin 子目录，如 C:\MinGW\bin（在安装时记住 MinGW 的安装路径），保存后重新启动命令行，gcc 就应该可以正常工作了。

A.3.2 常见编译选项

先建立一个 test.c，试试常见的编译选项。

```
#include<stdio.h>
main()
{
    int a, b;
    scanf("%d%d", &a, &b);
    int c = a+b;
    printf("%d%d\n", c);
}
```

编译一下，命令为 gcc test.c。程序没有输出，代表一切均好。检查目录（Windows 下用 dir，Linux 下用 ls），会发现多了一个 a.exe（Windows）或 a.out（Linux），这就是程序的编译结果。

gcc test.c-o test 命令会让编译出的可执行程序名为 test.exe（Windows）或 test（Linux）。这样，就能用 test（Windows）或 ./test（Linux）方式运行程序。

也许读者已经看出了上述代码中的一些问题，不过当程序更加复杂时，人眼就不一定能快速找到错误了。在这样的情况下，编译选项能起作用：gcc -test.c -o test -Wall。这次，编译器指出了 3 个警告：main 函数没有返回类型、没有返回值、printf 的格式字符串可能有问题。还可以进一步用 -ansi-pedantic，它会检查代码是否符合 ANSI 标准（-ansi 只是判断是否和 ANSI 冲突，而 -pedantic 更加严格）。它进一步指出了上述代码中的另一个问题：ANSI C 中不允许临时声明变量，而必须在语句块的首部声明变量。

在 C 语言中，另一个常用的编译选项是 -lm，它让编译器连接数学库，从而允许程序使用 math.h 中的数学函数。C++ 编译器会自动连接数学库，但如果程序的扩展名是 .c，且不连接数学库，有时会出现意想不到的结果。

另一个有用的选项是 -DDEBUG，它在编译时定义符号 DEBUG（可以换成其他，如 -DLOCAL 将定义符号 LOCAL），这样，位于 #ifdef DEBUG 和 #endif 中间的语句会被编译。而在通常情况下，这些语句将被编译器忽略（注意，不仅是不会执行，连编译都没有进行）。

可以用 -O1、-O2 和 -O3 对代码进行速度优化。一般情况下，直接编译出的程序比用 -O1 编译出的程序慢，而后者比 -O2 慢。尽管理论上 -O3 编译出的程序更快，但由于某些优化可能会误解程序员的意思，一般比赛中不推荐使用。另外，如果你的程序中有一些不确定因素（如使用了未初始化的变量），运行结果可能会和编译选项有关——用 -O1 和 -O2 编译出的程序也许不仅是速度有差异，答案甚至都有可能不同！当然，这种情况出现的前提是程序有瑕疵。如果是一个规范的程序，运行结果不会和优化方式有关。